

En la arquitectura moderna con Flutter, estas herramientas son el estándar indiscutible para modelar estados de la interfaz (*Loading*, *Success*, *Error*) sin condicionales anidados y para reutilizar código de transformación de datos de forma ultralimpia.

13.1 Genéricos (<T>) y Restricciones de Tipo (*Bounded Generics*)

Los genéricos permiten escribir clases, interfaces o métodos que operan sobre tipos de datos abstractos definidos al momento de instanciarse, garantizando la seguridad de tipos (*type-safety*) en tiempo de compilación.

Restricción de Tipos (*extends*)

Podemos acotar el tipo genérico para que únicamente acepte clases que deriven de una superclase específica mediante la palabra reservada *extends*:

```
// T solo puede ser de tipo num o subclases (int, double)
T sumar<T extends num>(T a, T b) {
    return (a + b) as T;
}
```

13.2 Métodos de Extensión (*extension on*)

Las extensiones nos permiten agregar nuevas funcionalidades a clases o bibliotecas existentes **sin modificar su código fuente original ni recurrir a la herencia**.

```
// Extensión sobre la clase String del SDK de Dart
extension ExtensionString on String {
    bool get esBoletaEscom {
        final regExp = RegExp(r'^\d{10}$');
        return regExp.hasMatch(this);
    }
}

void main() {
    print('2026630001'.esBoletaEscom); // true
}
```

13.3 Novedades de Dart 3: *sealed class* y *Pattern Matching*

Clases Selladas (*sealed class*)

Una **sealed class** es una clase abstracta cuya jerarquía de subclasses está ****estrictamente restringida al mismo archivo .dart**** donde fue definida. Esto le permite al compilador realizar un **análisis exhaustivo** (*exhaustiveness checking*).

Pattern Matching y Expresiones switch

Combinadas con **sealed class**, las expresiones **switch** de Dart 3 garantizan que cubramos **todos los casos posibles** de un estado sin necesidad de escribir cláusulas **default**. Si agregamos un nuevo estado en el futuro, el compilador marcará un error de inmediato si no es manejado.

```
sealed class EstadoUI {}
class EstadoCargando extends EstadoUI {}
class EstadoExito extends EstadoUI { final String datos; EstadoExito(this.datos); }
class EstadoError extends EstadoUI { final String mensaje; EstadoError(this.mensaje); }

// Expresión Switch con Pattern Matching
String renderizar(EstadoUI estado) => switch (estado) {
  EstadoCargando() => 'Cargando datos...',
  EstadoExito(:final datos) => 'Datos recibidos: $datos',
  EstadoError(:final mensaje) => 'Error detectado: $mensaje',
};
```

13.4 Ejemplo

Creen el archivo **bin/d13_generics_dart3.dart** en VS Code. El siguiente programa muestra cómo envolver respuestas asíncronas con genéricos y procesar los estados de UI usando **sealed class** y *Pattern Matching*.

```
/// Módulo D13: Generics, Extensions y Dart 3 (Sealed Classes & Pattern Matching)
/// Demostración de wrappers reutilizables, extensiones de utilidad y manejo de estados exhaustivos.

void main() {
  print('=====');
  print('  ARQUITECTURA DART 3 Y ESTADOS UI (D13) - ESCOM');
  print('=====');

  // 1. Uso de Métodos de Extensión sobre tipos primarios
  print('--- 1. Métodos de Extensión ---');
  const String correoAlumnos = 'OZZY@ESCOM.IPN.MX';
  print('Texto original: $correoAlumnos');
  print('¿Es correo institucional IPN?: ${correoAlumnos.esCorreoIpn}');

  // 2. Simulación de Estados de UI usando Sealed Classes y Pattern Matching
  print('--- 2. Procesamiento Exhaustivo de Estados con Switch Expression ---');
  final List<ResultadoApi<String>> listaEstados = [
```

```

    Cargando(),
    Exito('Datos de perfil recuperados correctamente.'),
    Fallo('No hay conexión con el servidor central.'),
];

for (final estado in listaEstados) {
  final String vista = procesarEstadoUi(estado);
  print(' [ESTADO UI] -> $vista');
}

print('\n=====');

/// Extensión personalizada sobre la clase String
extension ValidacionesString on String {
  bool get esCorreoIpn => toLowerCase().endsWith('@ipn.mx') ||
    toLowerCase().endsWith('@escom.ipn.mx');
}

/// Jerarquía Sellada (Sealed Class) para control estricto de estados
sealed class ResultadoApi<T> {}

class Cargando<T> extends ResultadoApi<T> {}

class Exito<T> extends ResultadoApi<T> {
  final T datos;
  Exito(this.datos);
}

class Fallo<T> extends ResultadoApi<T> {
  final String mensajeError;
  Fallo(this.mensajeError);
}

/// Función que utiliza Pattern Matching de Dart 3 sobre la Sealed Class
String procesarEstadoUi(ResultadoApi<String> estado) {
  return switch (estado) {
    Cargando() => 'Mostrando indicador de carga (Spinner)...',
    Exito(:final datos) => 'Renderizando vista con payload: "$datos"',
    Fallo(:final mensajeError) => 'Desplegando diálogo de error: "$mensajeError"',
  };
}

```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d13_generics_dart3.dart
```

13.5 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para dominar las extensiones, la restricción de genéricos y las clases selladas.

Ejercicio 1: Extensión de Formato de Moneda y Validación Numérica

Crea utilidades de formato mediante extensiones:

1. Crea una extensión `FormateadorNum` sobre el tipo `double`:
 - Agrega el getter `String get aMonedaMxn` que devuelva el número formateado con el símbolo de peso y a dos decimales (ej. `250.5.aMonedaMxn` -> `"$250.50 MXN"`).
 - Agrega el getter `bool get esPositivo` que devuelva `true` si el valor es mayor a cero.
2. En `main()`, prueba la extensión con distintas variables de tipo `double`.

Ejercicio 2: Repositorio Genérico y Jerarquía de Autenticación Dart 3

Escribe un modelo de estado de sesión para Flutter:

1. Define la `sealed class EstadoAutenticacion`.
 2. Define las subclasses:
 - `NoAutenticado`
 - `Autenticando`
 - `Autenticado` que contenga un campo genérico `T usuario`.
 - `ErrorAutenticacion` que contenga una propiedad `String error`.
 3. Escribe una función `evaluarSesion(EstadoAutenticacion estado)` que utilice una expresión `switch` con *pattern matching* para imprimir el mensaje correspondiente según el estado activo.
 4. Demuestra su ejecución en `main()` iterando sobre todos los estados.
-